Optimizing the RK3588 NPU Export Graph

Batch-major attention and broadcast condition injection for fused classifier-free guidance (CFG)

This report documents an export-only optimization of the AeroDM trajectory diffusion network. The model weights, attention equations, guidance formula and 60-step sampling schedule were retained. The change targets the computation graph produced by PyTorch-to-ONNX export and subsequently compiled by RKNN.

Measured outcome

Measurement	Original / reference	Optimized
One CFG prediction pair	16.53 ms: original batch=2	6.67 ms: optimized batch=2
Sequential CFG prediction pair	9.62 ms: two batch=1 calls	6.67 ms: optimized batch=2
Complete guided trajectory	704.5 ms: sequential CFG	555.7 ms: optimized fused CFG

In the same interleaved forward benchmark, the optimized export reduced latency by **59.7%** relative to the original batch=2 graph and by **30.7%** relative to two batch=1 calls. In the complete sampler, it reduced guided latency by **21.1%** relative to sequential CFG.

What “batch=2” means here

The trajectory batch remains one. At each diffusion step, sequential CFG makes one conditional and one style-null network call. Fused CFG combines those two inputs into a single network batch of two, then splits the predictions and applies the same CFG equation. The style-null branch uses a zero action vector while retaining the goal and history.

```
prediction = style_null + scale * (conditional - style_null)
scale = 2.0
```

Important scope boundary

The optimized graph agrees with the original FP32 model within the tested tolerance, but the FP16 NPU trajectories are not numerically identical. Across 96 paired samples, the maximum relative L2 difference from sequential CFG was **6.66%**. Latency gains therefore do not establish equivalent collision avoidance or target-reaching performance.

Evidence: saved RKNN layer profiles, export-validation output, interleaved forward timings and paired full-sampling results. No external benchmark numbers are used.

Finding the bottleneck

The diagnosis used the compiled RKNN per-layer execution reports. For each operator, the report exposes its type, execution target, input/output shapes, elapsed time and graph name. Exact graph names identify comparable nodes; operator-family totals help when compiler fusion changes node boundaries.

```
OpType | Target | InputShape | OutputShape | Time(us) | FullName
```

The relevant comparison is batch=2 versus two batch=1 calls

Operator family	B=1 (us)	B=2 (us)	B=2 - 2 x B=1
Transpose	60	2627	+2507 us
Reshape	240	1841	+1361 us
All convolution variants	1352	3204	+500 us
LayerNorm / exNorm	566	1411	+279 us
Attention / exSDPAttention	1324	2362	-286 us
Expand	21	770	+728 us
All operators	3987	13089	+5115 us

Transpose count rose from 3 to 37. Together, Transpose and Reshape accounted for **3868 us, or 75.6%**, of the excess over two batch=1 profile totals. The attention kernel itself took less time than twice the batch=1 attention time. This points to graph layout handling as a major avoidable cost.

A concrete shape change

The first decoder LayerNorm took 46 us with compiled input **(1,128,1,80)**, but 535 us with **(80,128,1,2)**. Here 80 is the combined sequence length and 128 is the feature dimension. Several projection operators also received the latter arrangement. These observations motivated the batch-major export rewrite.

CPU execution was present, but localized

CPU-attributed time increased from 67 to 856 us. The condition-vector Expand changed from NPU execution at 21 us to CPU execution at 770 us, accounting for 90.0% of the batch=2 CPU time. Other CPU-labelled nodes were input/output and small reshape/transpose operations. Attention, normalization and projections remained on NPU.

These are single profiled-forward measurements, not repeated wall-clock averages. Shape reports do not expose the complete physical memory layout, strides or internal tiling. They support a layout diagnosis, not a claim about undocumented hardware internals.

Change 1: explicit batch-major attention

The original model uses TransformerDecoder with batch_first=True. That API setting alone does not guarantee that the exported and compiled graph maintains the same layout internally. The observed batch=2 graph contained costly conversions around attention and projection operations.

A dedicated evaluation wrapper, BatchMajor, reuses the existing network modules and parameters. It replaces the decoder call with explicit attention and layer operations, keeping the batch axis separate from the sequence axis throughout the attention computation.

Stage	Shape / operation
Input and projected tokens	[B,S,D], with B=2, S=80 and D=128
Projection into attention heads	[B,S,H,d] -> [B,H,S,d], H=4 and d=32
Scaled dot-product attention	softmax(Q K^T / sqrt(d) + causal_mask) V
Merge attention heads	[B,H,S,d] -> [B,S,H,d] -> [B,S,D]
Output projection	Original attention output-projection weights

```
q = linear(query, Wq, bq).reshape(B, S, H, d).transpose(1, 2)
k = linear(memory, Wk, bk).reshape(B, M, H, d).transpose(1, 2)
v = linear(memory, Wv, bv).reshape(B, M, H, d).transpose(1, 2)
z = scaled_dot_product_attention(q, k, v,
                                 attn_mask=mask, dropout_p=0.0)
z = z.transpose(1, 2).reshape(B, S, D)
output = original_output_projection(z)
```

Preserving the decoder semantics

The Q/K/V weights and biases are sliced from each existing attention module’s packed projection parameters. Self-attention uses the current layer state for query, key and value. Cross-attention uses the current state as query and the fixed, conditioned input sequence as memory, matching the original decoder.

```
memory = conditioned_input
for layer in decoder_layers:
    z = norm1(z + self_attention(z, z, causal_mask))
    z = norm2(z + cross_attention(z, memory, causal_mask))
    z = norm3(z + linear2(activation(linear1(z))))
```

The wrapper retains the positional encoding, causal masks, post-norm residual order, feed-forward activation, optional final normalization and final 60-token selection. Dropout is disabled because this is an evaluation export. The implementation asserts that the decoder is not configured for pre-norm.

This is a static batch=2 exporter for the tested obstacle-encoder-disabled model. It is not a generic replacement for all Transformer configurations or a training implementation.

Change 2: broadcast condition injection

The original forward explicitly expands a per-sample condition embedding along the sequence dimension before adding it to the token representation. This operation is mathematically simple, but the original batch=2 RKNN graph executed the Expand on CPU.

Original export path

```
# condition: [B,D]; tokens: [B,S,D]
condition_sequence = condition.unsqueeze(1).expand(-1, S, -1)
tokens = tokens + condition_sequence
```

Optimized export path

```
# Broadcast [B,1,D] over the sequence dimension
tokens = tokens + condition.unsqueeze(1)
```

Both expressions add the same condition vector to every token of a given sample. Broadcasting lets the compiler handle that addition without retaining the explicit Expand node. The optimized compiled report confirms that the Expand disappeared. This is a result of the tested export and compiler, not a guarantee that every backend will optimize broadcasting identically.

Observed compiled-graph changes

Profile metric	Original B=2	Optimized B=2
Transpose count	37	6
Transpose time	2627 us	149 us
Reshape time	1841 us	111 us
Explicit Expand time	770 us	Removed
CPU-attributed total	856 us	142 us
Total operator time	13089 us	6124 us

The remaining CPU-labelled operations are input/output and small shape/layout transformations. The optimized graph still contains transpose and reshape operators; the goal was to eliminate costly layout handling, not to remove every shape operation.

What was deliberately retained

No retraining, weight update, INT8 quantization, reduced diffusion-step count or changed guidance strength was introduced. History and current inputs use the original projection; their positional encoding and condition embedding remain intact. The sampler still combines conditional and style-null predictions using the existing CFG formula.

The two graph changes were evaluated together. The measurements establish the benefit of the combined export, but do not separately quantify how much speedup each change would provide in isolation.

Timing protocol and numerical checks

Forward-call benchmark

The test ran on RK3588 with three NPU cores, FP16 network execution, CPU affinity on cores 4-7, OPENBLAS_NUM_THREADS=1 and OMP_NUM_THREADS=4. Each method received 20 warmups and 300 timed measurements in rotated order. Every measurement processed two inputs. Profiling was disabled; wall time included runtime input/output handling.

Method	Mean (ms)	Median (ms)	P95 (ms)
Two original B=1 calls	9.620	10.094	10.862
Original B=2 call	16.531	17.533	18.349
Optimized B=2 call	6.667	6.931	7.290

Complete 60-step sampling

The same checkpoint and 32 fixed scenes with 3-5 obstacles were tested three times, giving 96 samples per mode after two warmups. Trajectory batch was one. Mode order rotated; noise seeds were paired. CFG scale was 2.0, logistic guidance gamma was 1.0 and sigma was 0.3. Network execution used NPU FP16; diffusion updates and guidance used CPU FP32.

Mode	Mean trajectory latency
Unguided	338.434 ms
Sequential CFG	704.549 ms
Optimized fused CFG	555.714 ms

The 21.1% end-to-end reduction is smaller than the forward-call gain because the full loop also performs diffusion updates, guidance and host work. The previous 1292.97 ms original-fused result came from a different run; it is not a concurrent full-sampler control.

Validation: agreement depends on precision and scope

Comparison	Cases	Maximum observed error
Wrapper vs original PyTorch FP32	32	Absolute: 1.7881e-6
Exported ONNX vs original FP32	32	Absolute: 2.4438e-6
Optimized NPU vs original FP32 network	8	Relative L2: 3.5296%
Optimized NPU vs sequential NPU network	8	Relative L2: 0.1546%
Full fused vs sequential NPU trajectories	96	Relative L2: 6.6574%

FP32 assertions used atol=2e-5 and rtol=2e-4. Full-trajectory maximum absolute normalized-component error was 0.5595703; this is not an error in meters. All outputs were finite. Relative L2 is ||a-b||2 / max(||b||2, 1e-12), with b the reference for each comparison. Reported errors are maxima over cases, not averages.

Reproduction, artifacts and interpretation

Export and conversion

The trusted checkpoint is loaded into the existing architecture. Export uses ONNX opset 17 and fixed input shapes. RKNN conversion targets rk3588 with optimization_level=3 and do_quantization=False. The runtime used in the experiment was RKNN 2.3.2; the host PyTorch version was 2.2.0.

Remote working directory: /home/neardi/Project_rk3588/NPU/runs/cfg_fused_20261017

```
export OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=4
../../.venv/bin/python export_layout.py
../../.venv/bin/python convert_model.py \
  --onnx artifacts/layout_b2.onnx \
  --output artifacts/layout_b2.rknn
../../.venv/bin/python test_layout.py
../../.venv/bin/python benchmark_cfg_hardware.py \
  --backend npu --rknn-two artifacts/layout_b2.rknn \
  --output npu_layout.json
```

Evidence and implementation files

Local base: output/rk3588_cfg_20261017/ within the DTG_pytorch repository.

Relative path	Purpose
remote_results/artifacts/profile_b1.txt; profile_b2.txt	Original per-layer shape, target and timing evidence
layout_optimization/export_layout.py	Batch-major evaluation wrapper and FP32 validation
layout_optimization/profile_comparison.json	Name-matched nodes and operator-family totals
layout_optimization/remote_results/artifacts/profile_layout.txt	Optimized compiled per-layer report
layout_optimization/remote_results/layout_forward.json	Interleaved forward latency summary
layout_optimization/remote_results/npu_layout.json	Full-sampler timings and paired numerical errors
layout_optimization/remote_results/artifacts/layout_b2.rknn	Optimized experimental NPU model

Interpretation and next validation step

The evidence supports an export-layout bottleneck: extra layout operations and a CPU Expand made the original fused graph inefficient. The rewritten graph removed most of that overhead and made fused CFG faster than sequential CFG on the tested RK3588 setup.

Mathematical equivalence does not guarantee identical FP16 trajectories. The measured full-sampler difference requires a separate paired evaluation of collision-free rate, target reach and trajectory fidelity before treating the optimized model as interchangeable for planning. The 32-scene trajectory PDF illustrates actual NPU outputs but does not replace that quality evaluation.

Original model artifacts were retained. The optimized model is stored separately. This report describes the measured configuration only; it does not claim transferability to other batch sizes, obstacle encoders or RKNN versions.